

5 Indexace v prostorových databázích (specializace WDOP)

1. Definujte R-strom a vysvětlete, k čemu se používá. Popište strukturu vnitřních a listových uzlů a vysvětlete roli minimálních ohraničujících obdélníků, tj. MBR.
2. Vysvětlete na jednoduchém příkladu, jak se pomocí R-stromu vyhodnocuje dotaz "Najdi všechny objekty, které leží v zadaném obdélníku nebo ho protínají." Uveďte, kdy lze celý podstrom bezpečně vynechat, a kdy je naopak nutné pokračovat do více větví stromu.
3. Porovnejte R-strom s alespoň jednou další metodou prostorového indexování, například Quad-tree, k-d-tree, Z-křivkou nebo Hilbertovou křivkou. Uveďte hlavní výhodu a nevýhodu obou přístupů.

Nástin řešení 1. R-strom je výškově vyvážený vícecestný stromový index pro prostorová data. Je zobecněním B-stromu pro vícerozměrné objekty. Každý uzel obsahuje položky ve tvaru: MBR + odkaz.

- MBR (minimum bounding rectangle) je nejmenší osově zarovnaný obdélník, který ohraničuje daný objekt nebo všechny objekty v příslušném podstromu.

- V listových uzlech jsou uloženy MBR konkrétních prostorových objektů a odkazy na tyto objekty. Ve vnitřních uzlech jsou uloženy MBR potomků a odkazy na odpovídající podstromy.

2. Při vyhodnocení takového dotazu se strom prochází od kořene. Pokud se MBR daného podstromu se zadaným obdélníkem neprotíná, celý podstrom lze přeskočit. Pokud se protíná, pokračuje se níže. V listech se pak ověří konkrétní objekty.

3. Výhodou R-stromu je, že pracuje přímo s prostorovými objekty a hodí se i pro obecnější tvary. Nevýhodou je, že MBR různých větví se mohou překrývat, takže při dotazu může být nutné procházet více částí stromu.

Z-křivka je jiný způsob prostorového indexování. Převádí vícerozměrné souřadnice na jednu hodnotu, například tak, že se střídají bity souřadnic x a y. Nad touto jednorozměrnou hodnotou pak lze použít běžný index, například B+ strom.

Výhodou Z-křivky je jednoduchost a možnost využít klasické indexy. Nevýhodou je, že prostorová blízkost se zachovává jen přibližně. Dotaz na objekty v zadané oblasti se proto často musí rozložit na více intervalů v Z-pořadí a nalezené kandidáty je nutné dodatečně ověřit.

6 MapReduce (specializace WDOP)

Uvažujte kolekci textových dokumentů. Každý dokument má identifikátor `doc_id` a textový obsah. Cílem je vytvořit invertovaný index, tedy pro každé slovo zjistit, ve kterých dokumentech se vyskytuje a kolikrát.

Například z dokumentů:

D1: data data index D2: web data D3: index web web

má vzniknout:

data -> [(D1, 2), (D2, 1)] index -> [(D1, 1), (D3, 1)] web -> [(D2, 1), (D3, 2)]

1. Navrhněte řešení pomocí MapReduce. Uveďte, zda lze úlohu vyřešit jedním MapReduce krokem, nebo zda je vhodné použít dva kroky. Své rozhodnutí zdůvodněte.
2. Specifikujte alespoň v pseudokódu, co bude dělat funkce `map` a co bude dělat funkce `reduce`. Uveďte příklad emitovaných dvojic klíč–hodnota.
3. Vysvětlíte, zda by zde pomohla funkce `combine` a k čemu obecně slouží.

Nástin řešení 1. Úlohu lze vyřešit jedním MapReduce krokem. Mapper pro každé slovo v dokumentu vypíše jako klíč dané slovo a jako hodnotu identifikátor dokumentu. Reducer potom pro každé slovo dostane seznam dokumentů, ve kterých se slovo objevilo, a spočítá četnosti výskytů podle jednotlivých dokumentů. Použití dvou kroků není nutné, ale může být užitečné u velkých dat, pokud chceme nejprve zmenšit mezivýsledky, například spočítáním lokálních četností slov v dokumentech.

2. Funkce `map` zpracuje jeden dokument. Pro každé slovo v textu vytvoří dvojici, kde klíčem je slovo a hodnotou je identifikátor dokumentu: `map(doc_id, text):` pro každé slovo v textu: `emit(slovo, doc_id)`

Například z dokumentu D1: data data index vzniknou dvojice data -> D1, data -> D1 a index -> D1.

Funkce `reduce` dostane pro každé slovo všechny hodnoty, tedy seznam dokumentů, ve kterých se slovo vyskytlo. V tomto seznamu spočítá, kolikrát se opakuje každý dokument:

`reduce(slovo, seznam_doc_id):` spočítej četnosti jednotlivých `doc_id` `emit(slovo, seznam_dvojic (doc_id, četnost))`

Například ze vstupu data -> [D1, D1, D2] vytvoří výstup data -> [(D1, 2), (D2, 1)].

3. Funkce `combine` by zde pomohla. Obecně slouží jako lokální redukce provedená po mapperu ještě před přesunem dat k reducerům. Jejím cílem je zmenšit objem mezivýsledků posílaných po síti.

V této úloze může combiner lokálně sloučit opakované výskyty stejného slova ve stejném dokumentu. Například místo tří dvojic data -> D1, data -> D1, data -> D1 může poslat kompaktnější informaci data -> (D1, 3). Reducer pak z těchto částečných výsledků sestaví konečný seznam dokumentů a četností. Combiner nemění význam výpočtu, pouze snižuje množství přenášených dat.

7 Transakční rozvrhy (specializace WDOP)

Pro následující transakční rozvrh, kde R znamená operaci čtení a W operaci zápisu a ??? reprezentuje jednu neznámou databázovou operaci čtení nebo zápisu:

	T_1	T_2	T_3
1	W(<i>Project</i>)		
2		???	
3		W(<i>Project</i>)	
4		COMMIT	
5			R(<i>Customer</i>)
6			W(<i>Project</i>)
7			COMMIT
8	R(<i>Customer</i>)		
9	COMMIT		

1. Určete všechny databázové operace čtení nebo zápisu proměnné, které po dosazení do transakce T_2 místo ??? v čase 2 způsobí, že rozvrh nebude konfliktově uspořádatelný (conflict-serializable). Své rozhodnutí zdůvodněte.
2. Určete všechny databázové operace čtení nebo zápisu proměnné, které po dosazení do transakce T_2 místo ??? v čase 2 způsobí, že rozvrh nebude zotavitelný (recoverable). Své rozhodnutí zdůvodněte.

Nástin řešení Rozvrh bez operace ??? v čase 2 je jak konfliktově uspořádatelný, tak zotavitelný. V precedenčním grafu existují pouze hrany $T_1 \rightarrow T_2$ a $T_2 \rightarrow T_3$. Je tedy acyklický. Není zde ani žádné čtení nepotvrzených dat, které by spolu se špatným pořadím operací COMMIT mohlo způsobit nezotavitelnost.

1. Pouze operace W(*Customer*). V precedenčním grafu tak přibude hrana $T_2 \rightarrow T_1$ a vznikne cyklus s existující hranou $T_1 \rightarrow T_2$. Jiné operace R(*Customer*), W(*Project*) ani R(*Project*) novou hranu nepřidají. Stejně tak DB operace na jakékoli jiné proměnné.
2. Pouze operace R(*Project*). Vznikne tím nepotvrzené čtení hodnoty *Project*, zapsané transakcí T_1 , přičemž COMMIT transakce T_2 předchází operaci COMMIT transakce T_1 . Jiné operace W(*Project*), W(*Customer*) ani R(*Customer*) čtení nepotvrzených dat nezpůsobí. Stejně tak DB operace na jakémkoliv jiné proměnné.

8 Konceptuální modelování (specializace WDOP)

V rámci návrhu datového modelu aplikace byly na konceptuální úrovni identifikovány dvě datové třídy - *Hudebník* a *Nástroj*. O hudebníkovi je nutné si pamatovat jeho unikátní kombinaci jména, příjmení a roku narození. O nástroji je potřeba si pamatovat jeho unikátní název, a dále typ. Dále byly identifikovány následující vztahy:

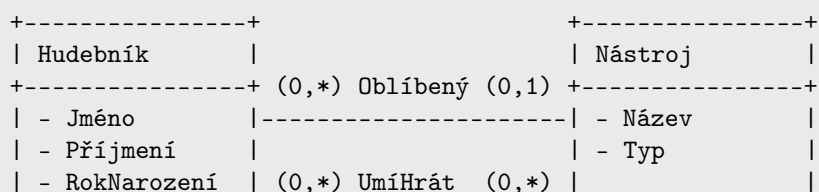
- Hudebník má nejvýše jeden *oblíbený* nástroj. Jeden nástroj může být oblíbený libovolným počtem hudebníků.
- Hudebník může *umět hrát* na libovolný počet nástrojů a naopak na jeden nástroj může umět hrát libovolný počet hudebníků. Je potřeba vědět, od kterého roku hudebník na nástroj hraje.

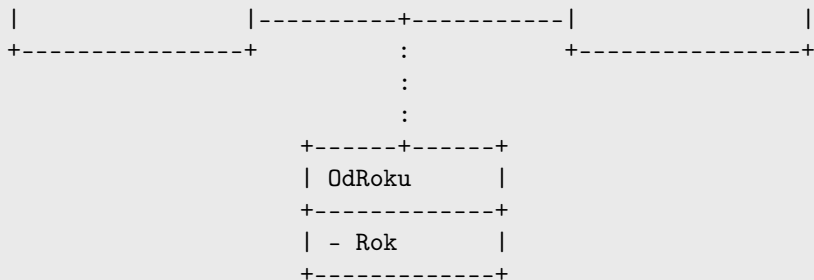
Pro výše popsanou situaci

1. Načrtněte pro popsanou situaci odpovídající UML nebo ER model. Nezapomeňte vyznačit kardinality vztahů.
2. Převeďte Vámi navržený konceptuální model na logický relační model. Nezapomeňte vyznačit všechny klíče a referenční integritu (cizí klíče). Převod proveďte tak, aby v cizích klíčích nemusela být nikdy vložena NULL hodnota.
3. Je logický relační model vzniklý převodem konceptuálního modelu vhodný z hlediska dosažené normální formy? Pokud ne, jak by měl být logický relační model upraven, aby vhodný byl a proč?

Nástin řešení

1. Řešení v UML:





2. – Hudebník(HudebníkID, Jméno, Příjmení, RokNarození)
 - Nástroj(NástrojID, Nazev, Typ)
 - Oblíbený(HudebníkID, NástrojID),
 $\text{HudebníkID} \subseteq \text{Hudebník.HudebníkID}$,
 $\text{NástrojID} \subseteq \text{Nástroj.NástrojID}$
 - UmíHrát(HudebníkID, NástrojID, Rok),
 $\text{HudebníkID} \subseteq \text{Hudebník.HudebníkID}$,
 $\text{NástrojID} \subseteq \text{Nástroj.NástrojID}$

Kardinalita vztahů se projeví v rozdílné definici klíčů vztahových tabulek.

3. Jediné funkční závislosti jsou ty na klíčích, a tedy je výsledné schéma v BCNF. Není potřeba nic dalšího dělat.